M2104 Object-Oriented Design & Modelling – Course Text – Explaining a Program
Session 1

1 Lesson : object-oriented design & modelling

1.1 Base concepts

Design and modelling have been contrasted for a while: philosophical tradition attributes to Plato the vision of *ideas* that would be matrices generating everything that exists, and his disciple Aristotle viewed them as *categories*, product of the human mind that seeks similarities in its environment. In similar fashion, we can view the preparation of an object-oriented program as a *creative* activity in which the programmer is the author: the classes they write are models for the objects, which are their instances. We speak of *design*. We can also view that preparation of an object-oriented program as an *analytic* activity, in which the programmer is an analyst that researches the behaviours of a physical phenomenon, a business process in a company, or the mechanics of a fictive world described to them by the scenarist of a video game: the programmer aims to represent using classes the different characteristics of the objects they encounter. We speak of *modelling*. In fact, design and modelling are most often used together, and the dialogue between those two approaches makes it possible to converge towards a full specification of a model using objects and classes of the program to be realised or in progress.

1.2 Goals for this course

The goals for this course are, on one hand, to provide a report on your choices as a software developper and, on the other hand, to prepare full specifications that can be used as a basis for software projects.

1.3 Means and grades

This course is entirely made of classes integrating lessons and exercises. It will be graded using a class grade, mostly consisting in the evaluation of one homework assignment (in pairs, weighted 3), one written test (short exam, weighted 2) and a final exam (weighted 5). The M2104 course itself is weighted 2,5 in the second semester.

1.4 Objects and classes

In the object-oriented paradigm of programming, *objects* denote units in programs that:

- provide services to other objects (the *methods*): we speak of operations, processes, or behaviours, *what the object do* or *what it is used for*,
- contain data on which they operate (the *attributes* or fields).

Objects are defined by *classes* (in Java, a class – explicit or not – is necessary in order to create an object, which is not the case in all languages). An object is an *instance* of a class. The relationship between classes and objects is that which exists between types and individuals (natural integers are a type, 7 is an individual member of that type; `String` is a Java class, in `String s="Hello"; s` is an object, instance of that class). Object-oriented programming in Java consists in defining in classes the behaviour that the objects of each class will have (their *instance methods*), the data on which they depend (their *instance attributes*), as well as the operations and data shared in the class (the *class* methods and attributes). This means first to report the *organisation* of these classes: which are their operations, data, and relationships between one another.

1.5 The language used

In order to report on the choices made during software development, and to communicate efficiently between members and teams of software design, development, programming, testing, maintenance… but also with clients and external contractors, a specific language must be used. UML, the Unified Modelling Language, is the one used in this course; it is one of the most mainstream modelling languages used in the industry, based on several diagrams allowing to specify many facets of software projects, information systems, or business processes. Our first objective is to agree on the basics of a few UML diagrams; before making this language yours and using it by adapting its syntax to your own uses and that of your professional environment, we insist that you conform to the elements of syntax given here – we will therefore act as in the beginning of the teaching of a new language by correcting form mistakes, in order for you to perfectly master the basics.

1.6 Basis of the class diagram

The first UML lexeme we will cover is the *class diagram*: its goal is to represent classes being part of a program, their content (partially or in full), i. e., the operations (methods) and data (attributes) defined in each class (whether of instance or of class), and more importantly the relationships between the classes: the biggest draw of a precise class diagram is to enable the easy visualisation of those relationships, and thus which class uses what other class.

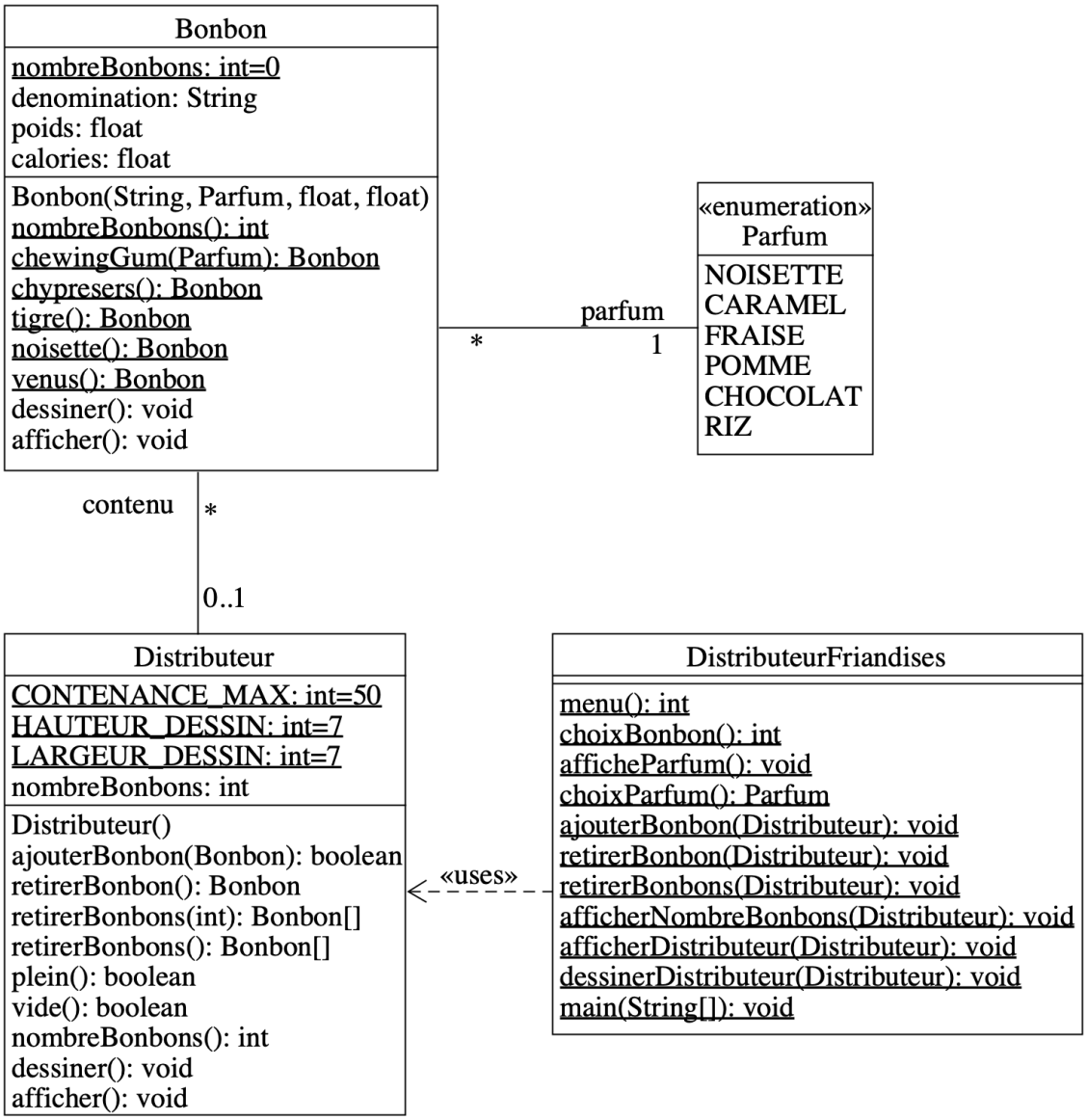
A class is represented by a rectangle box detailing:

- its name,
- the list of its attributes,
- the list of its methods.

Depending on what the diagram has to represent, some of these can be left out (some or all attributes, some or all methods): the most simple representation of a class is a rectangle box containing its name.

In the first practical of the M1103 module on classes and object-oriented programming, the concepts of instance and class methods and attributes were illustrated using the classes *Bonbon* (candy) and *distributeur* (vending machine).

Here is the (original in French) class diagram corresponding to that practical:



The enumeration *Parfum* (flavour – a specific kind of class in Java) and the main class *DistributeurFriandise*, which has no instance (it has no attribute and only class methods), are apart. Attributes are noted with their name and type (after the column character, `:`). The methods include their names and the list of types of their parameters (the name of those parameters can also appear in the class diagram, but are often left out as in here for simplicity's sake), followed by the return type (`void` for actions). The class methods and attributes (`static`) are underlined (the rest are instance methods and attributes). The constants are written in all caps (like in Java), and attributes can have an initial value signalled with = (after their type).

The most important concerns the relationships between classes. A simple continuous line denotes an *association between classes*. An association corresponds to an (instance) attribute in the Java code of a class, the type of which is another class. **Warning!** Associations never appear in the attribute list of a class in the class diagram, but are clearly marked by this link: the interest of the diagram is to allow to visualise immediately those associations. Associations are decorated with mandatory *multiplicities* : the character `*` denotes *as much as needed*, a specific integer number *n* (often 1), *exactly n*, and `i..j` a possible interval between *i* and *j* (included).

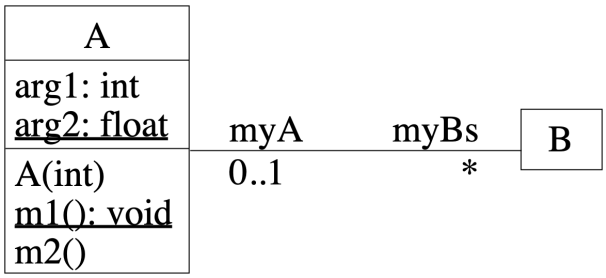
Contrary to other schemas, multiplicities are placed on the side of the association link nearest to the object having that multiplicity (there is one flavour per candy, in this diagram). Associations are also decorated by roles (at least one) that characterise the association (it is the name of the corresponding attribute in Java code).

Also, there can be *usage* relationships between classes. This happens when in a class A, the code of at least one of its methods (or constructors) calls a method or creates an object of a class B, it is noted that *A uses B* with a dashed line ended by an arrow, decorated with the *uses* stereotype. Here, the main class creates an object of type *Distributeur* in its `main` method.

1.7 Summary

Classes provide services by the means of operations: the methods, acting on pertinent data: the attributes, whether of class or of instance; in the latter case, instances represent different entities of a single type, that perform the same services. The class diagram makes it easy to visualise all or part of the classes in the program (their operations and data), as well as the relationships between those classes (associations and use links).

Note, on the following diagram: a *data type*, a *class attribute*, an *instance attribute*, a *class method*, a *constructor*, a *class name*, an *association*, a *multiplicity*, a *role*.



2 Lesson: Visibility

Attributes of an objects are most of the time *the data that it requires to function*: an object *encapsulates* data and operations in order to perform said operations. The end goal is to be able to *call some methods from outside of the object*.

In order to ease the separation between those methods and *what is only needed for the object itself*, different *visibilities* are used. The visibility of a method or attribute defines its scope, and thus the fact of being useful for *itself* or for *others*. Java, as well as UML, define four visibility levels:

Visibility	Java keyword	UML symbol	Explanation
Package	(none)	~	Default visibility in Java, the attribute/method is accessible in the *package*. The goal is to share data and operations within a small, coherent set of classes.
Public	`public`	+	Full publicity: provided to the outside, generally a service provided by the class in the form of a public method. For example, the `main` method must be public in order to act as an *entry point* for the program.
Private	`private`	−	Private scope: internal to the class, only methods of this class may use this in their code. Most of the time, attributes are private (read-only access to the attributes can sometimes be provided by methods called *accessors* or `get`-methods, but this is not always advisable). Some methods, some constructors can also be private.
Protected	`protected`	#	Used in specific circumstances that will be detailed later.

3 *Exercise – Java code transcription*

In this course, we will often study Java code *without the body of methods and constructors* (more details on that in the programming course). This is the declaration of a class *Drink*, thus without code. **Exercise**: give the class diagram (one class here) that corresponds to this code, with types and visibilities.

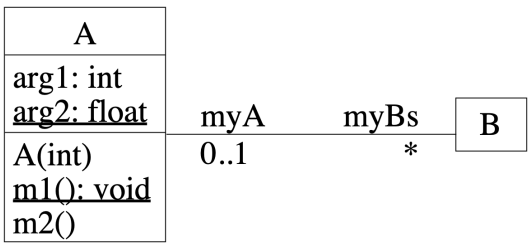
```
/**
 * Class allowing to create drinks for a vending machine
 */
public class Drink {
    public static final String VOLUME_UNIT ="mL";
    private String denomination;
    private int volume;
    private double calories;
    private boolean withGas;
    private String flavour;
    /**
     * Parameter'd constructor
     * @param myDenomination the commercial name for the drink
     * @param myVolume the bottle/can volume in mL
     * @param myEnergy the total energy in calories
     * @param isWithGas the fact to contain carbonic acid or not
     * @param myFlavour a description of the flavour of the drink
     */
    public Boisson(String myDenomination, int myVolume, double myEnergy,
                   boolean isWithGas, String myFlavour) {
    }
    /**
     * Method giving the number of calories per 100 mL.
     * @return energy/100mL
     */
    public double energyPerHundredMl() {
    }
    /**
     * Method giving the fact that this drink is with gas or not
     * @return accessor to "with gas"
     */
    public boolean getGas() {
    }
    /**
     * Factory method creating a standard sparkling soda
     * (uses the class constructor)
     * @param sodaName the commercial name of the soda
     * @param theFlavour a description of the flavour of the soda
     * @return a soda, new class instance
     */
    public static Drink makeSoda(String sodaName, String theFlavour) {
    }
}
```

4 *Lesson: Associations and containers*

4.1 *Review: association towards several objects*

Recall that associations **must** appear in the class diagram as links and not in the list of attributes, even if, in the Java code, they are attributes referring another class; the clear representation of relationships between classes remains the goal of the class diagram. For associations in which multiplicities are `1` or `0..1`, the code is simple: it is a reference to an object, that can be null or not. But in the case of multiplicities higher than one or open like `*`, things are different.

In the case of an association between classes as in the previous diagram:



Class *A* refers to *a collection* of objects of type *B* (named *myBs*). In Java, an array can be used (the Java code of class A will then contain an attribute `B[] myBs`), this is what was used un the *candy vending machine* practical (each vending machine has an array of candies). But arrays have strong limitations, and in recent languages, such as Java (for a long time now), we prefer to use collections, or, sometimes, associative mappings: the *containers*.

4.2 Collections

A *Collection* is an object representing several objects, most of the time of a single type. Arrays are collections. In Java, we can also use lists (sequences) and sets.

4.2.1 Lists

Lists in Java are represented using the general type `List`. One of its implementations is `ArrayList`: objects of type `ArrayList` behave like arrays (we can instantly access any item using its index between 0 and $n-1$ for n items), and are easier to change (the size of an `ArrayList` can be changed at will, items can be added at the beginning, end, in the middle, replaced, deleted…). Moreover, the Java API provides many methods on those objects, ensuring their ease of use and performance.

This is a short example of code using this class:

```
ArrayList<Drink> drinks = new ArrayList<>();
  // Parameter'd type + type inference
drinks.add(Drink.makeSoda("TuringSoda", "Cherry")); // Addition at the end
drinks.add(Drink.makeSoda("AdaSoda", "Cola")); // Addition at the end
drinks.add(1, Drink.makeSoda("KolmogorovSoda",
  "Bergamot")); // Addition at specific index
System.out.println(drink.get(2)); // Read-only access to index
System.out.println(drink.size()); // Access to the size (number of items)
drinks.set(0, new Drink("NoetherJuice",
  500, 450.0, false, "Lime")); // Modification to index
```

4.2.2 Sets

Sets are non-indexed collection in which objects appear only once; these are useful data structures when there is no specific order (first-last) between the items. In Java, the general type `Set` represents sets. Two implementations are `TreeSet` and `HashSet`: the internal structure of the data differs but the end result is similar. This is an example of a lexicon (set of words):

```
TreeSet<String> lexicon = new TreeSet<>();
  // Parameter'd type + type inference
lexicon.add("cat"); // Addition
lexicon.add("dog"); // Addition
lexicon.add("recursive"); // Addition
lexicon.add("anamorphosis"); // Addition
lexicon.remove("dog"); // Removal - uses "equals"
if (lexicon.contains("cat") { // Test - uses "equals"
  System.out.println("Meow.");
}
for (String s : lexicon) { // Iteration over all items
  System.out.println(s); // Here in word order (lexicographic)
}
```

4.3 Associative mappings

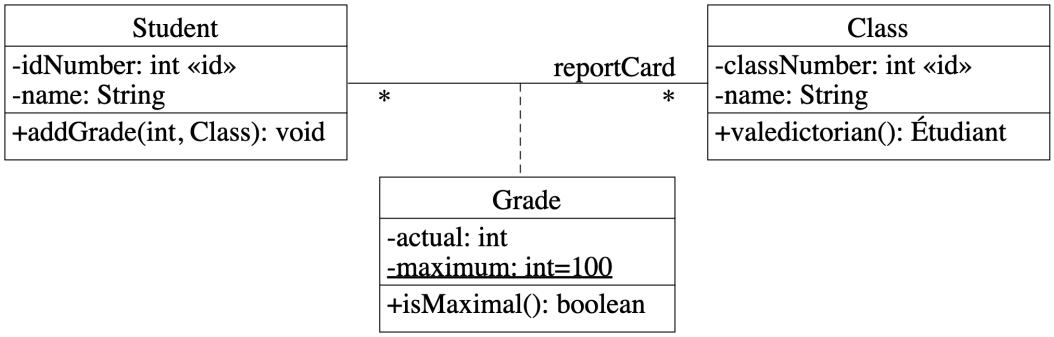
Associative mappings or dictionaries represent plural associations: we may consider them as arrays in which the index is an object instead of an integer between 0 and $n-1$ (for n-length arrays). Associative mappings thus associate a *key* (unique, corresponding to an index in an array or a keyword in a dictionary) to a *value* (the item stored in the array, or the definition in the dictionary). They are not collections, as they do not contain a set of objects of one type, but two (the one for keys and the one for velues).

In Java, associative mappings are represented using the general type `Map`. As for `Set`, two main implementations are provided: `HashMap` and `TreeMap`. This is an example of an address book:

```
HashMap<String, String> addressBook = new HashMap<>();
  // Two types: key, value
addressBook.put("Mr. Johnson", "489, Elm Street");
  // Addition: key, value
addressBook.put(Ms. Marvel", "75, Sesame Street");
addressBook.put("Jane Doe", "999, Unknown Lane");
System.out.println(addressBook.get("Mr. Johnson"));
  // Access at a given key
if (addressBook.containsKey("Ms. Marvel")) {
  // Key test - uses "equals"
  System.out.println("ok");
}
addressBook.replace("Jane Doe", "16, Ash Lane");
  // Replacing a value associated to an existing key
for (String name : addressBook.keySet) {
  // Access to all keys (as a set)
  System.out.format("%s : %s",
    name, addressBook.get(name));
 // Access to each value, using each key
}
```

4.4 Association classes in UML

A class *A* containing an associative mapping having keys of type *B* and values of type *C* can denote a specific relationships between the objects of those three classes: in UML, this can be an *association class* (other data structures can also represent that kind of relationships). *A* and *B* are bound in such a way that each object of type *A* has access to several objects of type *B*, and that each object of type *B* has access to several objects of type *A*. For example, in a software used for handling students' grades, each student is enrolled in several classes (courses) and for each class, several students are enrolled in it. But for each pair of *A* and *B*, there is one single object of type *C* characterising their relationship. In that example, for each student and each class, there is a grade that this student has got in that class. In Java code, we may have in a *Student* class an attribute such as `HashMap<Class, Grade> reportCard;`. In UML, *Grade* is an *association class* written like this:



An association class is this represented on the association it characterises, linked to it by a dashed line.

4.5 Class diagram and information systems

Most situations studied in this course are based on object-oriented *programs* (apps), but class diagrams can also be used for *information systems*, or for programs based on information systems and/or databases. In such cases, marking the *identifier* for the objects can be useful (this corresponds to the *primary key* in a table). In UML, the identifier is marked with the *id* stereotype. In most association classes, the identifier for an object is the *couple of the identifiers of the objects it is linked to* (the grade of the class 2104 for the student of id 777 has $(777, 2104)$ as an id).

5 Exercise – Java code transcription

This is the declaration of a few Java classes, without the code for methods and constructors. **Exercise**: give the class diagram (four class including an association class) for this code, with types and visibilities.

```
/**
 * A virtual person in a simulated environment
 */
public class Person {
    private String name;
    private String profile;
    private Avatar representation;
    /**
     * Standard parameterized constructor.
     * @param name An unique identifier
     * @param profile Can be empty
     * @param representation The avatar shown on screen
     */
    public Person(String name, String profile, Avatar representation) {}
    /**
     * Changes the public profile (set method).
     * @param newProfile The text of the profile to set
     */
    public void setProfile(String newProfile) {}
    /**
     * Connects to a new server.
     * @param server The URI to connect to.
     */
    public void connect(String server) {}
    /**
     * Attempts to interact with a virtual pet.
     * @param p The interaction target
     * @return True if successful (depends of the friendliness of the pet)
     */
    public boolean interact(Pet p) {}
}

/**
 * A virtual pet (AI)
 */
public class Pet {
    static Set<Pet> allPets;
    private String name;
    private String species;
    private Avatar representation;
    private Map<People, Affinity> friends;
    /**
     * Standard parameterized constructor.
     * @param name An unique identifier
     * @param species E. g. "cat", "dog", "ant"...
     * @param representation The avatar shown on screen
     */
    public Pet(String name, String species, Avatar representation) {}
    /**
     * Factory method used to create new pets periodically.
     * @return a new random Pet
     */
    public static Pet spawn() {}
    /**
     * Call this method periodically for random actions.
     */
    public  void doSomething() {}
    /**
     * Method called when people try to interact with this pet.
     * @param p the person trying to interact
     * @param true if successful, the person will be added as a friend
     *     or friendship will be upgraded
     */
    public boolean interaction(Person p) {}
}
```

```
/**
 * A class representing how a pet feels towards a particular person.
 */
public class Affinity {
    private Pet myPet;
    private Person myPerson;
    private int friendliness=1;
    /**
     * Parameterized constructor with a base friendliness level of 1.
     * @param thePet the pet in this relationship
     * @param thePerson the person in this relationship
     */
    public Affinity(Pet thePet, Person thePerson) {}
    /**
     * Method called when an existing relationship is improved.
     */
    public void increase() {}
    /**
     * Access a textual representation of the friendliness value.
     * @return text such as "friendly"
     */
    public String friendLevel() {}
}

/**
 * A representation for people and pets on screen.
 */
public class Avatar {
    private String image;
    private int[] location=new int[2];
    /**
     * Standard parameterized constructor
     * @param image The image to show on screen (file path)
     */
    public Avatar(String image) {}
    /**
     * Updates the screen with this avatar.
     */
    public void show() {}
    /**
     * Moves the avatar.
     * @param dx The horizontal movement (-1 to +1)
     * @param dy The vertical movement (-1 to +1)
     */
    public void move (int dx, int dy) {}
}
```